IT112 Web System and Technologies
Phase 5: Displaying and Managing Data on the Frontend
Lab Report by SURVIVORS

Group Members:

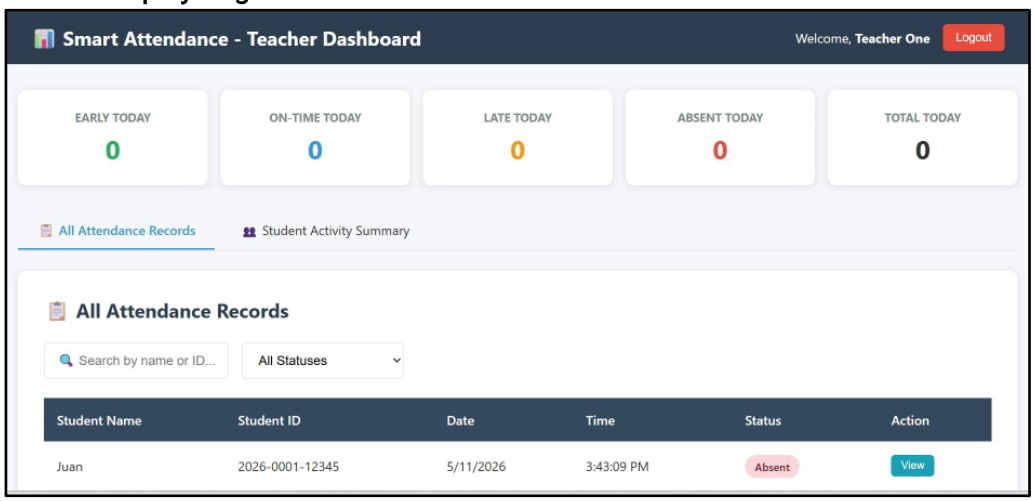
Jexson Sedon
John Roan Ballester
Michelle Diaz
Judy Pearl Balictar
Alyssa Jenille Reantaso
Ronel Garcia

Submitted to:

Guillermo V. Red,Jr,DIT
Assistant Professor IV

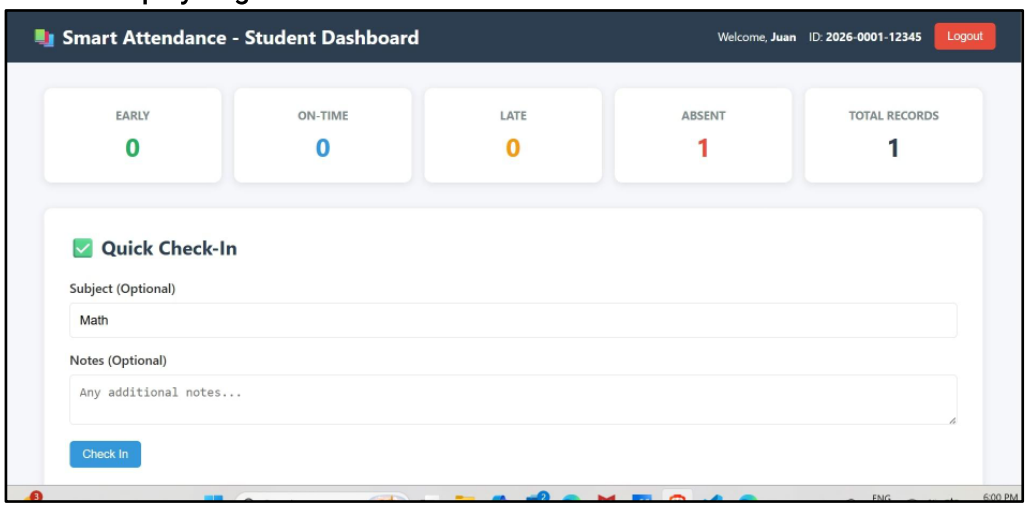
I. Screenshot of Display Pages

a. Display Page 1 – Teacher Dashboard



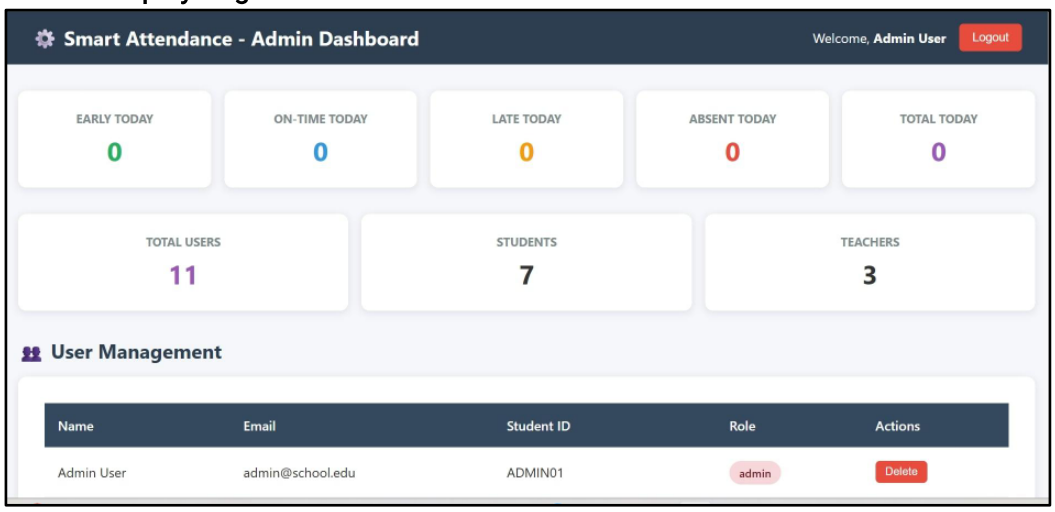
Notes:
This screenshot displays the Teacher Dashboard page connected to the backend database through GET API endpoints. The dashboard dynamically shows teacher-related information such as students info, schedules, or student records using JavaScript rendering techniques.

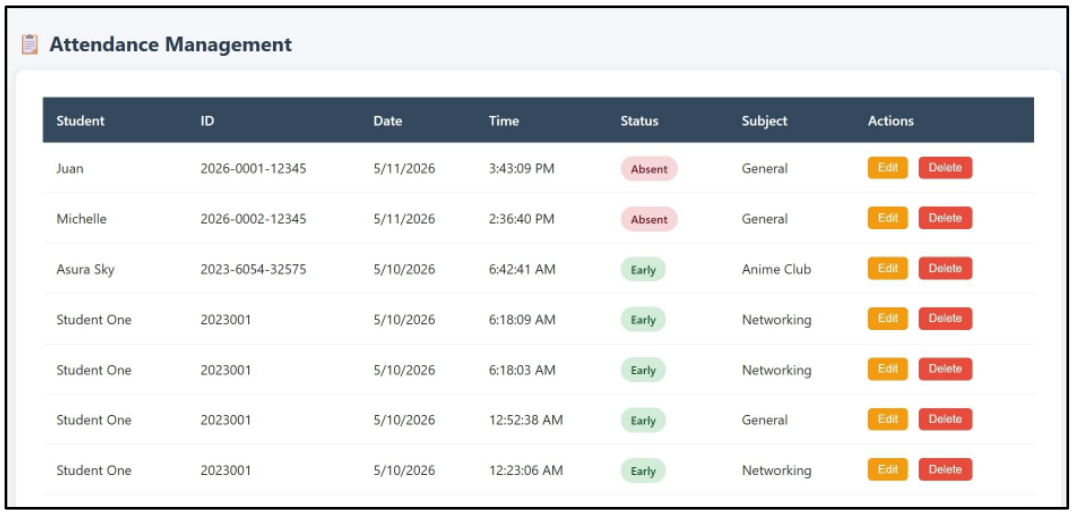
b. Display Page 2 – Student Dashboard

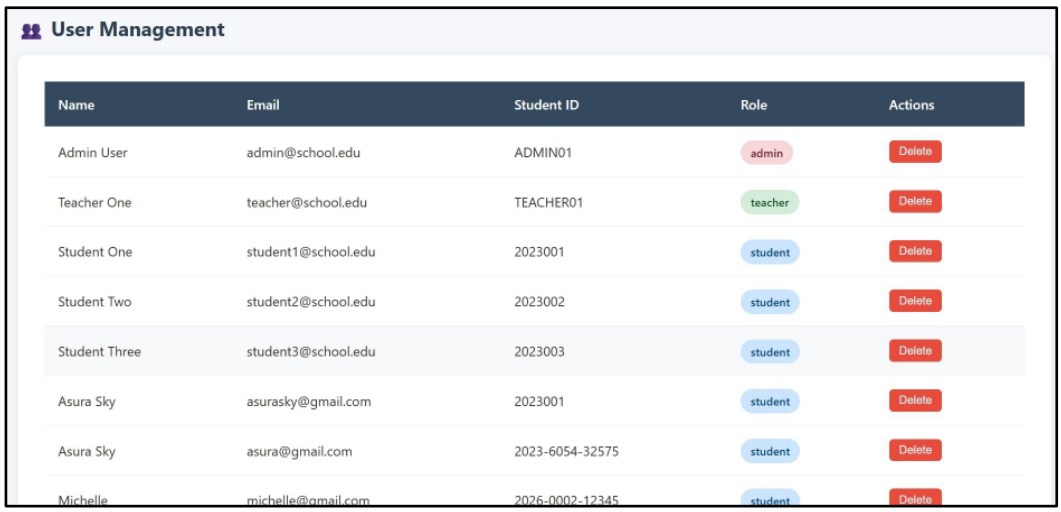


Notes:
This screenshot shows the Student Dashboard interface where student information is dynamically displayed using data retrieved from the backend API. JavaScript fetch() and DOM manipulation were used to render real-time student records, making the dashboard interactive and responsive.

c. Display Page 3 - Admin Dashboard

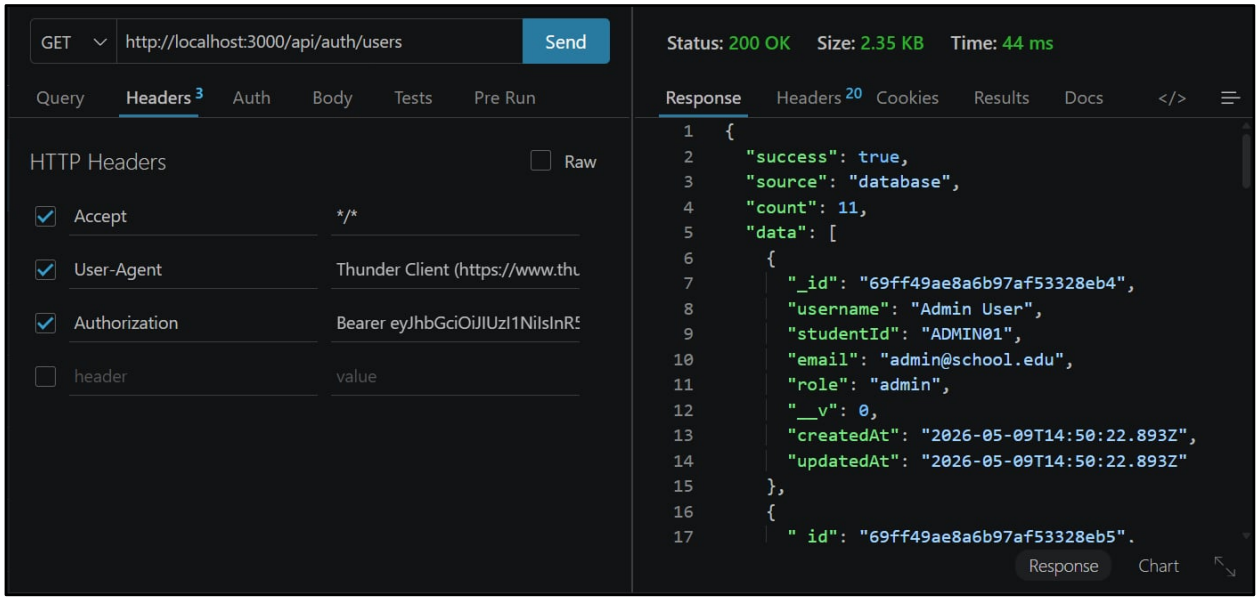






Notes:
This screenshot shows the Admin Dashboard where data from MongoDB is dynamically displayed on the frontend. The dashboard allows administrators to monitor and manage system records efficiently through real-time API integration and responsive UI components.

II. Sample Backend API Response



Notes:
This screenshot shows the JSON response returned by the backend GET API endpoint using Postman or Thunder Client. The response matches the schema stored in MongoDB.

III. Annotated JavaScript Snippet for Rendering Data

```
18
19  document.getElementById('logoutBtn')?.addEventListener('click', function() {
20    localStorage.removeItem('token');
21    localStorage.removeItem('currentUser');
22    window.location.href = 'login.html';
23  });
24
25  async function loadUsers() {
26    const tableBody = document.getElementById('usersTableBody');
27    if (!tableBody) return;
28
29    tableBody.innerHTML = '<tr><td colspan="5" class="text-center">Loading...</td></tr>';
30
31    try {
32      const response = await fetch('http://localhost:3000/api/auth/users', {
33        headers: {
34          'Authorization': `Bearer ${token}`
35        }
36      });
37
38      const data = await response.json();
39
40      if (!response.ok) {
41        tableBody.innerHTML = `<tr><td colspan="5" class="text-center text-danger">${data.message}</t
42        return;
43      }
```

```
135       const [attendanceRes, usersRes] = await Promise.all([
136         fetch('http://localhost:3000/api/attendance/stats', {
137           headers: { 'Authorization': `Bearer ${token}` }
138         }),
139         fetch('http://localhost:3000/api/auth/users', {
140           headers: { 'Authorization': `Bearer ${token}` }
141         })
142       ]);
143
144       const attendanceData = await attendanceRes.json();
145       const usersData = await usersRes.json();
146
147       if (attendanceData.data) {
148         document.getElementById('statEarly').textContent = attendanceData.data.today.Early;
149         document.getElementById('statOnTime').textContent = attendanceData.data.today['On-Time'];
150         document.getElementById('statLate').textContent = attendanceData.data.today.Late;
151         document.getElementById('statAbsent').textContent = attendanceData.data.today.Absent;
152         document.getElementById('statTotalToday').textContent = attendanceData.data.total;
153       }
154
155       if (usersData.data) {
156         document.getElementById('statTotalUsers').textContent = usersData.data.length;
157         const students = usersData.data.filter(u => u.role === 'student').length;
158         const teachers = usersData.data.filter(u => u.role === 'teacher').length;
159         document.getElementById('statStudents').textContent = students;
160         document.getElementById('statTeachers').textContent = teachers;
161       }
```

Notes:
This JavaScript snippet is responsible for fetching user data from the backend API using the fetch() method. The retrieved JSON data is processed and dynamically rendered into the HTML table by updating the DOM elements. It also includes authentication headers for secure API access and loading indicators for better user experience.

IV. GitHub Commit Logs

Notes:
The GitHub commit logs below display the updates made during Phase 5, including dashboard rendering, API integration, and frontend improvements. Due to unstable internet connection in their area, the GitHub Manager was unable to perform repository tasks regularly, so the Backend Manager temporarily handled the commits and synchronization of project files to maintain the group's workflow and progress.

V. Reflections of Each Member

Project Manager (Jexson Sedon)
This phase improved our coordination in handling frontend and backend integration. Managing dynamic rendering tasks helped us work more efficiently as a team.

Frontend Manager (Judy Pearl Balictar)
I learned how to retrieve data from APIs and display it dynamically using JavaScript and DOM manipulation. This activity improved my understanding of responsive dashboard design.

Backend Manager (John Roan Ballester)
I learned how important properly structured GET endpoints are for frontend integration. Testing API responses also improved my debugging skills.

Database Manager (Michaels Diaz)
This activity helped me understand how stored MongoDB data is retrieved and displayed on the frontend. I also learned how schemas affect displayed information.

GitHub Manager (Alyssa Jenille Reantaso)
I improved my skills in organizing commits and merging frontend/backend updates properly. GitHub collaboration became more important as the project structure grew.

Documentation Officer/Debugger (Ronel Garcia)
I learned how to properly document real-time rendering workflows and organize technical screenshots clearly. This phase also improved my attention to detail during testing and debugging.